

Reproduction of GPU Value Iteration for Perishable Inventory Control

Bin Zeng (Ben Tseng)

Email: bentseng06@gmail.com

REQUIRED LINKS

Executed Kaggle notebook: Perishable Inventory GPU Reproduction, saved version 356144063

REPRODUCTION OBJECTIVE

This report reproduces the main numerical and computational claims in Ortega, Hendrix, and García (2019). The paper studies an average-reward Markov decision process for perishable inventory. Products are tracked by remaining shelf life, demand is Poisson, older inventory is issued first, and a second product may be substituted when the first is unavailable. The original work accelerates the expensive portion of value iteration with CUDA. The reproduction has three goals: verify the one-product example, reconstruct the two-product state spaces and policies despite several apparent typographical inconsistencies, and compare CPU and GPU value iteration on one current platform.

The implementation is independent rather than a transcription of the unavailable source code. NumPy provides the CPU reference and CuPy evaluates the same batched Bellman update on an NVIDIA GPU. Both backends share the same state enumeration, transition events, immediate rewards, stopping rule, and numerical precision.

MODEL AND DECISION SEQUENCE

At the beginning of each day, the retailer observes the age distribution of inventory and selects an order quantity. Existing inventory is sold during the day under FIFO issuing. The order selected today appears as the freshest inventory in the next state, matching the paper's transition convention. For a state I , action q , stochastic event d , transition F , and value vector W , the implemented one-product update is

$$V(I) = s E[\text{sales} \mid I] + \max_q \{ E[W(F(q, I, d))] - cq \}.$$

For two products, the action is (q_a, q_b) and the update subtracts $c_a q_a + c_b q_b$. If

product B demand exceeds available B inventory, each unmet unit independently requests A with probability γ . Original A demand and accepted substitution demand are then served jointly from A. Poisson tail mass is retained by capping sales at available stock rather than truncating and discarding high-demand outcomes.

Relative value iteration stops when the span of successive differences is below 10^{-4} :

$$\text{span}(V - W) = \max_j (V_j - W_j) - \min_j (V_j - W_j) < 10^{-4}.$$

The lower and upper successive-difference bounds bracket the average daily reward; their midpoint is reported as the value-iteration profit estimate.

INTERPRETATION OF APPARENT TYPOGRAPHICAL ERRORS

The author advised that the paper may contain typographical errors and that the reproduction should proceed with a documented interpretation. Table 1 records the choices made here. They are treated as reproduction assumptions, not as claims about the authors' unpublished implementation.

Table 1: Paper text and reproduction treatment.

Paper text	Reproduction treatment
Eq. (1): $G^{-1}((s - c)/c)$	The standard critical fractile is $G^{-1}((s - c)/s)$. The printed expression equals one for $s = 1, c = 0.5$ and has no finite Poisson quantile. The corrected formula is noted, but it is not used to infer the dynamic-programming truncation. The paper's stated one-product maximum order of 9 is used directly.
Bellman Eqs. (8), (10), and (11) use $\min$	Algorithms 1 and 2 use $\max$, and the objective is profit maximization. All implemented Bellman updates therefore use $\max$.
Eq. (9): $\sum_{d=1}^Y dp_d$	Demand above stock still sells all Y units. The implementation includes the Poisson tail, equivalently $E[\min(d, Y)]$.
Table 3 and $N = (Q_a + 1)^M(Q_b + 1)^M$	Table entries Q_a, Q_b are interpreted as counts of admissible quantity levels. Values therefore range from 0 through $Q - 1$. This exactly reproduces P2–P4 state counts.
P1 has 14,461 states	Replaced by $11^4 = 14,641$; the published number appears to transpose 4 and 6.
P1 profit is 5.509	Not used as a numerical target. Under the stated means, prices, and costs, even complete fulfillment with zero waste gives an expected-profit upper bound of $(5 + 5)(1 - 0.5) = 5$.

IMPLEMENTATION

The state is an age-bucket tuple. With shelf life $M = 2$, two products require four inventory coordinates $(I_{a1}, I_{a2}, I_{b1}, I_{b2})$. For each state, the code evaluates every action pair and every capped demand/substitution event. Event probabilities and transition indices are computed once on the CPU. Bellman evaluation is then processed in state batches to keep memory bounded.

The GPU path uses CuPy-generated CUDA kernels for array gathering, probability-weighted reduction, reward addition, and maximization. It does not reproduce the paper’s hand-written CUDA 8 kernel line by line. This difference is intentional: the objective is to reproduce the model and assess current GPU execution while retaining a directly comparable CPU implementation.

The main source files are:

- `model.py`: parameter presets, state counts, probability events, FIFO transitions, and simulation;
- `solve.py`: CPU/CuPy Bellman sweeps, relative value iteration, policy extraction, and timing;
- `run.py`: command-line entry points for counts, one-product checks, two-product solving, simulation, and benchmarks;
- `tests/test_model.py`: probability, transition, count, and numerical regression tests.

ENVIRONMENT AND RUN INSTRUCTIONS

Local correctness checks used Apple ARM64, Python, and NumPy 2.0.2. The GPU experiments used a Kaggle “GPU T4 x2” session with Python 3.13.15 and CuPy 14.0.1. The program selected CUDA device 0, so one Tesla T4 executed the reported GPU work. CPU and GPU benchmark times were collected in the same Kaggle session using double precision, batch size 64, and 100 Bellman sweeps.

The paper used two Intel Xeon E5-2620v2 processors, two NVIDIA K80 cards, Ubuntu 16.04, CUDA 8, and an optimized C/CUDA program. Absolute seconds across the two environments are therefore not directly comparable.

From the project directory, the essential commands are:

```
python3 -m unittest discover -s tests -v
python3 run.py counts
python3 run.py one --life 2 --max-order 9 --backend cpu
```

```
python3 run.py two --case P1 --backend cuda --tol 1e-4
python3 run.py bench --backend cuda --iters 100 --batch 64
python3 run.py bench --backend cpu --iters 100 --batch 64
```

CORRECTNESS RESULTS

All five unit tests passed. They verify that stochastic event probabilities sum to one, FIFO transitions behave as intended, Table 3 counts are reconstructed, the one-product paper value is recovered, and a small two-product instance agrees across implementations.

Table 2: One-product reproduction for $M = 2$, $\mu = 5$, $s = 1$, $c = 0.5$, and maximum order 9.

Measure	Paper	Reproduction
Average daily profit	2.215	2.215160
Largest action selected by the policy	7	7
Iterations to span tolerance 10^{-4}	12	14

The two-iteration difference is sensitive to value initialization and normalization. The economically relevant value and the largest selected order agree with the paper.

Table 3: Two-product quantity ranges and reconstructed state counts for $M = 2$.

Case	μ_a, μ_b	A range	B range	States
P1	5, 5	0–10	0–10	14,641
P2	5, 6	0–10	0–12	20,449
P3	6, 6	0–12	0–12	28,561
P4	7, 7	0–13	0–13	38,416

For P1, GPU value iteration converged in 13 iterations and 2.8061 seconds. The final average-reward interval was $[4.5031745, 4.5032455]$, with midpoint 4.5032100. A 400,000-day simulation using seed 7643 produced average daily profit 4.5041738, A waste 6.0019%, and B waste 4.1794%. The paper reports waste of 5.97% and 4.14%, respectively. The close waste rates support the reconstructed policy dynamics even though the paper’s stated profit is infeasible under its own parameter values.

CPU/GPU BENCHMARK

Table 4 compares 100 fixed Bellman sweeps. This avoids convergence-count differences and follows the timing protocol described for the paper’s Table 4. CPU and GPU profit values agreed within 7.11×10^{-15} in all four cases.

Table 4: Paper and Kaggle runtimes for 100 Bellman sweeps.

Case	States	Paper CPU (s)	Paper GPU (s)	Kaggle CPU (s)	T4 GPU (s)	T4 speedup
P1	14,641	357.89	91.29	710.09	20.16	35.22×
P2	20,449	698.20	136.84	1,284.57	30.30	42.39×
P3	28,561	1,358.04	214.71	2,602.89	59.24	43.94×
P4	38,416	4,241.66	361.49	5,135.97	103.84	49.46×

The corresponding Kaggle 100-sweep profit estimates were 4.5032031, 5.0036025, 5.5087173, and 6.5183400 for P1 through P4. The full notebook cell required 9,958.844 seconds because the four NumPy CPU runs dominated total time.

The reproduction shows the same qualitative scaling conclusion as the paper: GPU benefit increases as the state/action space grows. The measured T4 speedups are larger than the paper’s total-runtime speedups because the software baselines differ. The original baseline was optimized sequential C, whereas the reproduction deliberately uses a readable NumPy reference. Consequently, 35–49× should be interpreted as the acceleration of this implementation on one Kaggle session, not as a cycle-for-cycle reproduction of the original CUDA experiment.

LIMITATIONS AND OPPORTUNITIES

The main limitation is that the original source code is unavailable and the author’s response does not resolve the notation or reported-value discrepancies. The adopted interpretation is strongly supported by the P2–P4 counts and the P1 waste rates, but it cannot prove that every low-level transition matches the unpublished implementation. The current GPU backend also uses generated array kernels rather than the state-specific shared-memory reduction described in Algorithm 5. CPU timing is therefore a correctness baseline, not a claim of best available CPU performance.

Several extensions would strengthen the study:

- publish an unambiguous symbol table, complete parameter files, source code, and exact event ordering;
- compare against an optimized multithreaded C++ CPU baseline and report repeated-run uncertainty, transfer time, kernel time, and memory consumption separately;
- perform truncation sensitivity tests to show that increasing the inventory/action cap does not change the selected policy;
- profile and fuse GPU operations, test multi-GPU state partitions, and compare

CuPy with a custom CUDA kernel;

- extend to more products, longer shelf lives, bidirectional substitution, nonstationary demand, lead-time uncertainty, and joint pricing/order decisions;
- use the exact GPU solver as ground truth for approximate dynamic programming or reinforcement learning when enumeration becomes infeasible.

CONCLUSION

The reproduction verifies the paper's central computational claim: value iteration for substitutable perishable products maps effectively to a GPU, and the benefit grows with problem size. The one-product value and policy limit were recovered, the P2–P4 state counts were reconstructed exactly, CPU and GPU values matched to floating-point precision, and the P1 waste behavior closely matched the paper. At the same time, the reproduction identifies several inconsistencies that materially affect interpretation. Recording those assumptions is necessary for the results to be reproducible rather than merely similar.

GENERATED FILES

- `benchmark_summary.json`, `cpu_results.json`, and `gpu_results.json`;
- `p1_gpu_policy.npz` and `p1_gpu_validation.txt`;
- executed notebook `perishable-inventory-gpu-reproduction.ipynb`;
- source files, unit tests, `README.md`, and `REPRODUCTION.md`.